# UNIT V

## Worked Case Study and Student Practice Activity

## MongoDB, Database Operations and Cloud Deployment

| Course Details | Information |
|---|---|
| Course Title | Full-Stack Web Development |
| Course Code | 23CM4121 |
| Unit | Unit V: MongoDB and Deploying Applications |
| Solved Example | Student Management System |
| Student Practice | Faculty & Course Workload Management System |

# PART A. Comprehensive Question

## Case Study: Student Management System

A college academic department maintains student personal and academic records in physical registers and offline spreadsheets. The academic administration faces significant hurdles in tracking enrolled student cohorts, updating semester standings, calculating department-wise average CGPA, and identifying students needing academic attention. The institution wants a modern, secure, web-based Student Management System backed by a cloud database that provides real-time access across devices.

Design and develop a database-driven Student Management System using React, Node.js, Express.js, and MongoDB. Deploy the backend, frontend, and database on modern cloud platforms, making the full application accessible via a public web address.

The application must satisfy the following core requirements:

- Store complete student details including Roll Number, Full Name, Email Address, Department, Academic Year, Section, and CGPA.
- Create and manage a MongoDB Atlas cloud database ('student_db') and 'students' collection using an appropriate schema and validation.
- Perform full CRUD operations: add a student, retrieve and display all students, update existing student records, and delete records with confirmation.
- Write raw and driver-based MongoDB queries for document insertion, retrieval, modification, and removal.
- Implement MongoDB Aggregation pipelines to compute department-wise enrollment statistics and average CGPA metrics.

- Create database indexes (unique on roll number & email, secondary on department) for fast lookups and data integrity.
- Configure MongoDB Atlas securely, explaining cluster creation, user authentication, and network access (IP whitelisting).
- Deploy the backend API, frontend React interface, and MongoDB Atlas database with environment variables and reverse proxies, testing the live system.

# PART B. Worked Solution

## 1. Analyse the requirements

| Requirement | Solution |
|---|---|
| Student identity | A unique roll number (rollNumber) and institutional email uniquely identify each student. |
| Student details | name, email, department, year, section, and cgpa are stored with automatic audit timestamps. |
| Data storage | MongoDB Atlas cloud database: student_db; collection: students. |
| Application operations | React SPA issues async HTTP requests to Express API routes; Express reads/writes documents using Mongoose. |
| Analytics & Reports | MongoDB Aggregation calculates total student count and average CGPA grouped department-wise. |
| Performance & Integrity | Unique indexes on rollNumber and email; secondary index on department for fast filtering. |
| Cloud Deployment | MongoDB Atlas (AWS Mumbai ap-south-1) for DB; Vercel for both Vite frontend and Express backend services. |

## 2. Design the MongoDB document

A student record is represented as a BSON document in the 'students' collection. An example document structure is:

```
{
  "_id": "6af073872b7ccbcd7d53e3f9",
  "rollNumber": "A24126552085",
```

```
  "name": "JAMI SRAVAN KUMAR",
  "email": "sravan@college.edu",
  "department": "AI&DS",
  "year": 3,
  "section": "B",
  "cgpa": 8.56,
  "createdAt": "2026-11-08T13:05:11.277Z",
  "updatedAt": "2026-11-08T13:05:11.277Z"
}
```

Field specification and validation design:

| Field | Data Type | Constraints | Purpose |
|---|---|---|---|
| rollNumber | String | Required, Unique, Trimmed | Unique academic roll identifier (e.g. A24126552085). |
| name | String | Required, Trimmed | Full student name. |
| email | String | Required, Unique, Lowercase | Institutional contact email. |
| department | String | Required, Indexed | Academic department (AI&DS, CSE, ECE, IT, etc.). |
| year | Number | Required, Min: 1, Max: 4 | Current year of study. |
| section | String | Required, Uppercase | Classroom section division (A, B, C, D). |
| cgpa | Number | Required, Min: 0.0, Max: 10.0 | Cumulative Grade Point Average (0.00 to 10.00). |

Each document automatically receives an immutable '_id' (ObjectId) generated by MongoDB. The application relies on this '_id' as the primary routing parameter for editing and deleting individual student entries.

## 3. Create the database and collection

In MongoDB Shell (mongosh) or Atlas console, execute the following commands to create the database, collection, and insert initial records:

```
use student_db

db.createCollection("students")
```

```
db.students.insertMany([
  {
    rollNumber: "A24126552084",
    name: "Bhuvan Teja",
    email: "bhuvan.teja@college.edu",
    department: "AI&DS",
    year: 3,
    section: "B",
    cgpa: 8.70,
    createdAt: new Date(),
    updatedAt: new Date()
  },
  {
    rollNumber: "A24126552085",
    name: "JAMI SRAVAN KUMAR",
    email: "sravan@college.edu",
    department: "AI&DS",
    year: 3,
    section: "B",
    cgpa: 8.56,
    createdAt: new Date(),
    updatedAt: new Date()
  },
  {
    rollNumber: "23A91A0501",
    name: "Rahul Kumar",
    email: "rahul.kumar@college.edu",
    department: "CSE",
    year: 3,
    section: "A",
    cgpa: 8.40,
    createdAt: new Date(),
    updatedAt: new Date()
  }
])
```

## 4. Perform MongoDB CRUD operations

| Operation | MongoDB Query | Purpose |
|---|---|---|
| Create | db.students.insertOne({ rollNumber: "23A91A0402", name: "Ananya Reddy", email: "ananya@college.edu", department: "ECE", year: 2, section: "A", cgpa: 9.25 }) | Inserts a new student document. |
| Read | db.students.find({ department: "AI&DS" }) | Retrieves all students in the AI&DS department. |

| | | |
|---|---|---|
| Update | db.students.updateOne({ rollNumber: "A24126552085" }, { $set: { cgpa: 8.90, section: "A" } }) | Modifies academic details for a specific student. |
| Delete | db.students.deleteOne({ rollNumber: "23A91A0402" }) | Permanently removes the matching student document. |

Additional useful MongoDB operational queries:

```
// Retrieve all students sorted alphabetically by name
db.students.find({}).sort({ name: 1 })

// Find all students with CGPA >= 8.5 (Distinction scholars)
db.students.find({ cgpa: { $gte: 8.5 } })

// Sort all students by CGPA in descending order (Rank list)
db.students.find({}).sort({ cgpa: -1 })

// Find 3rd-year students in Section B
db.students.find({ year: 3, section: "B" })

// Total student count in the database
db.students.countDocuments({})
```

## 5. Create indexes

Indexes prevent full collection scans by storing an ordered B-tree representation of specified fields. A unique index guarantees no duplicate roll numbers or emails are ever entered, while a secondary index on department optimizes filtering and aggregation:

```
// Enforce unique constraints at database level
db.students.createIndex({ rollNumber: 1 }, { unique: true })
db.students.createIndex({ email: 1 }, { unique: true })

// Secondary index for department filtering and aggregation
db.students.createIndex({ department: 1 })

// View existing indexes
db.students.getIndexes()
```

## 6. Use aggregation for department-wise student summary

The administration requires enrollment distribution and academic quality metrics across departments. The aggregation pipeline groups records by '$department', counts total enrolled students with '$sum: 1', and calculates average CGPA with '$avg: "$cgpa"':

```
db.students.aggregate([
  {
    $group: {
      _id: "$department",
      totalStudents: { $sum: 1 },
      averageCGPA: { $avg: "$cgpa" }
    }
  },
  {
    $sort: { totalStudents: -1 }
  }
])
```

Expected aggregation result based on sample records:

| Department (_id) | Total Students | Average CGPA |
|---|---|---|
| AI&DS | 2 | 8.63 |
| CSE | 1 | 8.40 |

## 7. MongoDB Atlas Configuration, Database & Collection Setup, and Express Connection

MongoDB Atlas is MongoDB's fully managed multi-cloud database service. Storing student records in Atlas involves a sequence of architectural and security configurations to establish authenticated, TLS-encrypted communication with the Express.js backend.

### Step 1: Deploy a Cloud Database Cluster in MongoDB Atlas

1. Create an account on cloud.mongodb.com and create a project titled 'Student Management System'.
2. Build a Cluster: Select the M0 Free Tier (Shared Cluster). Choose a cloud provider and region closest to your users—in our case, AWS Mumbai (ap-south-1)—to achieve minimal query latency.
3. Name the cluster (e.g., 'Cluster0') and deploy. Atlas provisions three replica set nodes (one PRIMARY, two SECONDARY) across availability zones to provide high availability.

### Step 2: Configure Database User Authentication

1. Navigate to Database Access under the Security section.
2. Click 'Add New Database User'. Choose Password-based authentication (SCRAM-SHA-256).
3. Set a dedicated username (e.g., 'housequeen01_db_user') and generate a strong password.
4. Assign Database User Privileges: Select 'Read and write to any database' or grant granular readWrite permissions specifically on 'student_db'.
5. Click 'Add User' to commit the security credentials.

### Step 3: Configure Network Access & IP Whitelisting

MongoDB Atlas blocks all incoming connections by default via its cloud firewall. Incoming traffic must be explicitly permitted:

1. In the left navigation, select Network Access under Security.
2. For Local Development: Click 'Add IP Address' and add your workstation's public IPv4 address (e.g. 106.192.43.240/32).
3. For Cloud Serverless Deployments (Vercel): Cloud platforms like Vercel execute code in AWS Lambda serverless containers with dynamic, rotating IP pools. To allow Vercel serverless functions to connect, add CIDR 0.0.0.0/0 ('Allow Access from Anywhere'). Authentication is strictly safeguarded by user credentials and mandatory TLS encryption.
4. Confirm and wait until the status displays 'Active'.

### Step 4: Create Database and Collection

1. Navigate to Database > Clusters > Browse Collections in Atlas.
2. Click 'Create Database'. Enter Database name: 'student_db' and Collection name: 'students'.
3. Atlas creates the namespace 'student_db.students' ready for CRUD operations and indexes.

### Step 5: Obtain the MongoDB SRV Connection String

1. In Atlas, click the 'Connect' button on your cluster card.
2. Select 'Drivers' (Node.js version 5.5 or later).
3. Copy the standard connection string:
   mongodb+srv://<username>:<password>@cluster0.q6bnike.mongodb.net/?retryWrites=true&w=majority
4. Replace '<username>' and '<password>' with your database user credentials. If your password contains special characters (@, :, /, %), ensure they are percent-encoded.

### Step 6: Connect Express.js Backend via Mongoose ODM

1. Install required packages in the backend: npm install express mongoose cors dotenv.
2. Store the connection string securely in backend/.env:
   MONGODB_URI=mongodb+srv://<user>:<password>@cluster0.q6bnike.mongodb.net/
   DB_NAME=student_db
   PORT=5000
3. Configure Mongoose Connection in server.js:
   To support both local persistent servers and Vercel serverless cold-starts without connection leaks, implement connection pooling and state verification:

```
// Robust Mongoose connection logic for both local and serverless execution
const mongoose = require("mongoose");
require("dotenv").config();

async function connectDB() {
    // Reuse existing connection if already established
    if (mongoose.connection.readyState >= 1) return;

    await mongoose.connect(process.env.MONGODB_URI, {
        dbName: process.env.DB_NAME || "student_db"
    });
}

// Express middleware ensuring DB connection before routing API requests
app.use(async (req, res, next) => {
    try {
        await connectDB();
        next();
    } catch (error) {
```

```
        res.status(500).json({ error: "Database connection failed: " + error.message
});
    }
});
```

## 8. Develop the Express.js backend (Full Code)

Full production code of backend/server.js:

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const dns = require("dns");

// Ensure proper SRV record resolution across local networks
dns.setServers(["8.8.8.8", "8.8.4.4"]);

require("dotenv").config();

const app = express();

app.use(cors());
app.use(express.json());

// Connection caching for persistent and serverless cloud execution
async function connectDB() {
    if (mongoose.connection.readyState >= 1) return;
    await mongoose.connect(process.env.MONGODB_URI, {
        dbName: process.env.DB_NAME
    });
}

app.use(async (req, res, next) => {
    try {
        await connectDB();
        next();
    } catch (error) {
        res.status(500).json({ error: "Database connection failed: " + error.message
});
    }
});

app.use("/api/students", require("./routes/students"));

app.get("/api/health", (req, res) => {
    res.json({ message: "Student Management API is running!" });
});
```

```
async function startServer() {
    try {
        await connectDB();
        console.log("MongoDB Atlas connected successfully!");

        const PORT = process.env.PORT || 5000;
        app.listen(PORT, () => {
            console.log(`Server running on port ${PORT}`);
        });
    } catch (error) {
        console.log("CONNECTION ERROR:", error.message);
    }
}

if (require.main === module) {
    startServer();
}

module.exports = app;
```

API Endpoint Route Specifications:

| Method | Endpoint | Action & Description |
|---|---|---|
| GET | /api/health | Health-check endpoint confirming Express and DB responsiveness. |
| GET | /api/students | Retrieves all registered student documents. |
| GET | /api/students/:id | Retrieves a single student document by MongoDB _id. |
| POST | /api/students | Validates and creates a new student document. |
| PUT | /api/students/:id | Updates an existing student document using MongoDB _id. |
| DELETE | /api/students/:id | Deletes a student document using MongoDB _id. |
| GET | /api/students/analytics/department | Executes aggregation pipeline for department stats. |

## 9. Develop the React frontend (Vite)

The frontend is built using Vite + React. It features an executive academic dashboard, responsive student directory, search by name/roll number, filtering by department and academic year, and real-time department analytics.

frontend/src/services/api.js (Centralized API Client):

```
const RAW_URL = import.meta.env.VITE_API_URL || 'https://fslab-casestudy.vercel.app';
const BASE_URL = RAW_URL ? RAW_URL.replace(/\/$/, '') : '';

async function request(endpoint, options = {}) {
  const url = `${BASE_URL}${endpoint}`;
  const headers = { 'Content-Type': 'application/json', ...(options.headers || {}) };
  const response = await fetch(url, { ...options, headers });
  const data = await response.json().catch(() => null);
  if (!response.ok) {
    throw new Error(data?.error || data?.message || `HTTP ${response.status}: Request
failed`);
  }
  return data;
}

export const studentApi = {
  getAllStudents: () => request('/api/students'),
  getStudentById: (id) => request(`/api/students/${id}`),
  createStudent: (data) => request('/api/students', { method: 'POST', body:
JSON.stringify(data) }),
  updateStudent: (id, data) => request(`/api/students/${id}`, { method: 'PUT', body:
JSON.stringify(data) }),
  deleteStudent: (id) => request(`/api/students/${id}`, { method: 'DELETE' }),
  getDepartmentAnalytics: () => request('/api/students/analytics/department'),
  checkHealth: () => request('/api/health'),
};
export default studentApi;
```

## 10. Run and test the application locally

| Test Case | Action / Input | Expected Verification Result |
|---|---|---|
| TC1: Health Check | GET /api/health | HTTP 200 OK with status message. |
| TC2: Create Student | Submit student form with valid fields. | HTTP 201 Created; student record stored in Atlas and added to table. |
| TC3: Read Students | Load/refresh the dashboard. | HTTP 200 OK; all registered students retrieved and rendered. |

| | | |
|---|---|---|
| TC4: Update Student | Edit CGPA & section of student. | HTTP 200 OK; document updated and persisted in Atlas collection. |
| TC5: Delete Student | Confirm deletion dialog in UI. | HTTP 200 OK; document deleted from MongoDB Atlas collection. |
| TC6: Validation | Submit duplicate roll number or CGPA > 10. | HTTP 400 Bad Request; server rejects invalid payload with error text. |
| TC7: Aggregate Stats | GET /api/students/analytics/department | HTTP 200 OK; returns array of grouped counts and average CGPAs. |

## 11. Deploy the application using cloud platforms (Vercel)

The project is deployed on Vercel as a multi-service monorepo using casestudy/vercel.json. Vercel manages shared routing, TLS certificates, edge caching, and serverless execution for both frontend and backend under a single domain:

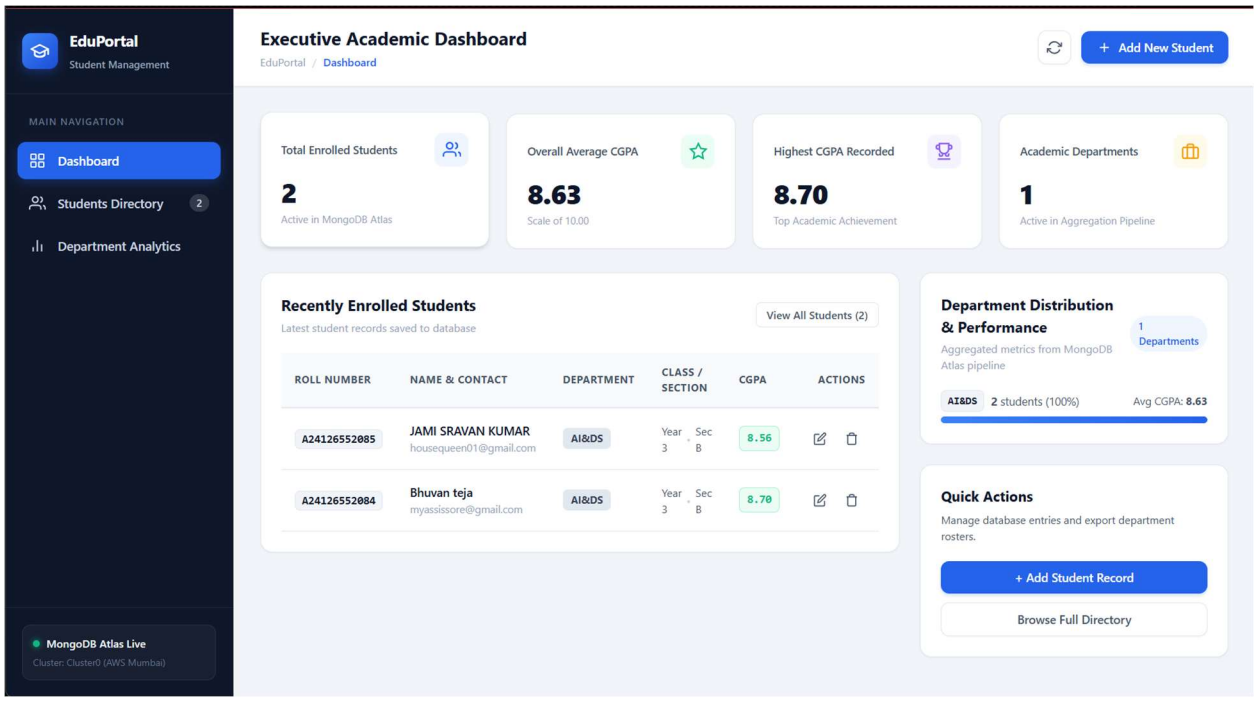
```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "services": {
    "backend": {
      "root": "backend",
      "framework": "express"
    },
    "frontend": {
      "root": "frontend",
      "framework": "vite"
    }
  },
  "rewrites": [
    {
      "source": "/api/(.*)",
      "destination": {
        "service": "backend"
      }
    },
    {
      "source": "/(.*)",
      "destination": {
        "service": "frontend"
      }
    }
  ]
}
```

Environment Variable Configuration Table:

| Variable Name | Target Environment | Purpose / Example Value |
|---|---|---|
| MONGODB_URI | Vercel Project (Backend Secret) | Atlas SRV connection string with credentials. |
| DB_NAME | Vercel Project (Backend) | student_db |
| VITE_API_URL | Vercel Project (Frontend) | https://fslab-casestudy.vercel.app/ |
| PORT | Local Development (.env) | 5000 (standalone Express port) |

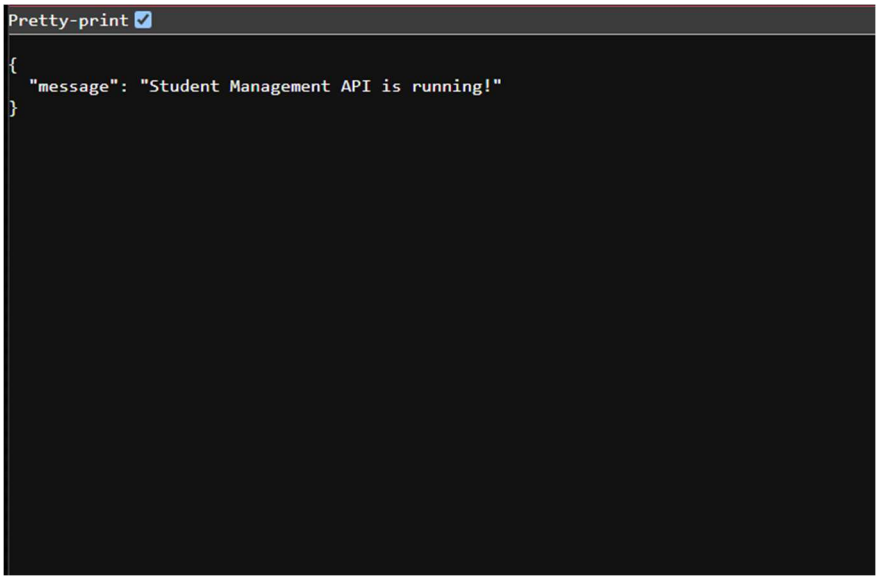
# PART C. Output Screen Shots (Test Case Results)

## Screen 1: Live Academic Dashboard (Deployed)



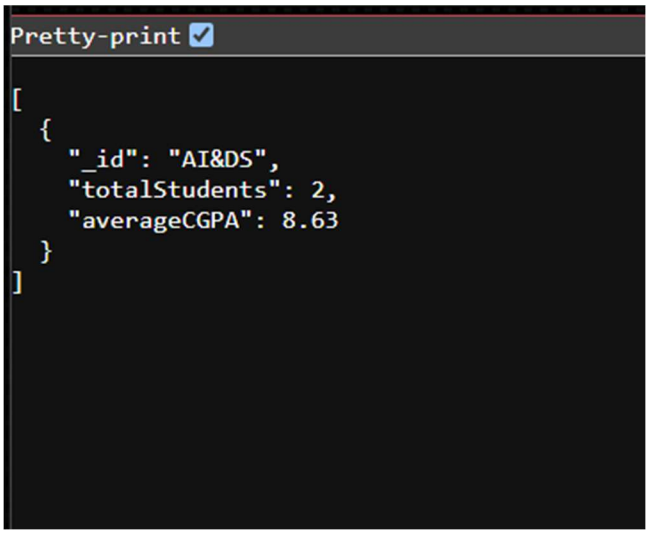

Displays the live Executive Academic Dashboard, key performance indicators (Total Enrolled Students: 2, Overall Average CGPA: 8.63, Academic Departments: 1), active MongoDB Atlas indicator (green dot), and student directory table.

## Screen 2: Backend API Health Check




HTTP 200 OK Response: {"message": "Student Management API is running!"}

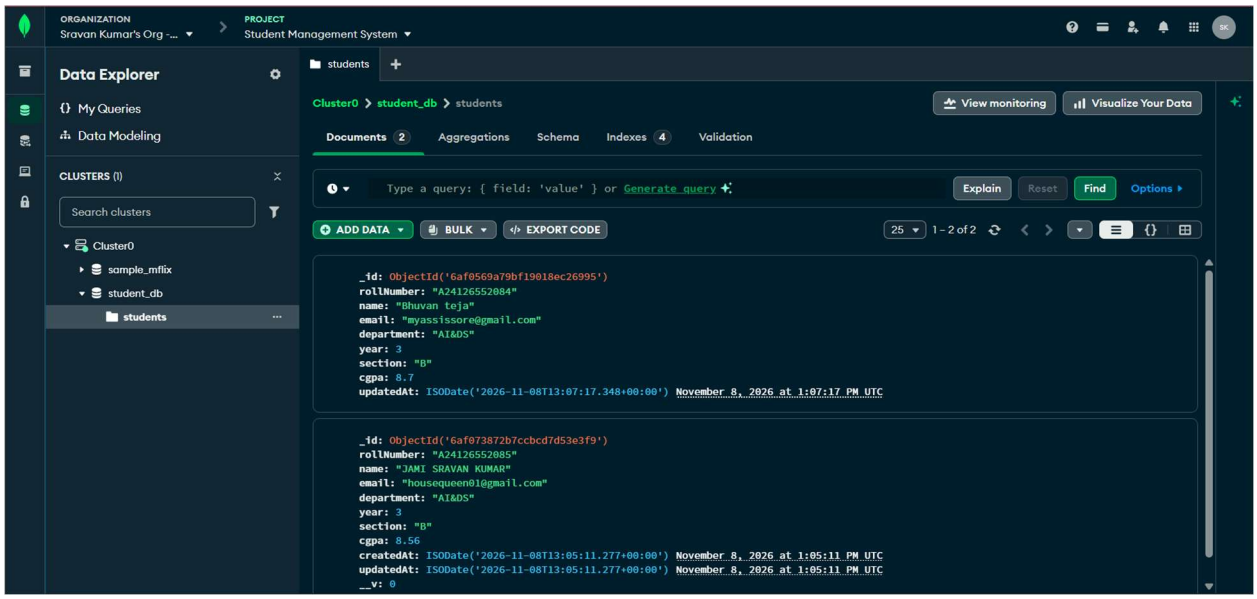
## Screen 3: Department-Wise Aggregation Report




HTTP 200 OK Response:

```
[
 {
  "_id": "AI&DS",
  "totalStudents": 2,
  "averageCGPA": 8.63
 }
]
```

## Screen 4: MongoDB Atlas Database Collection (students)




Shows persisted student records (Bhuvan Teja - Roll: A24126552084, JAMI SRAVAN KUMAR - Roll: A24126552085) with automatic timestamps.